MCMC SAMPLING FOR DUMMIES

STATS

AUTHOR
Thomas Wiecki

PUBLISHED
November 10, 2015

When I give talks about probabilistic programming and Bayesian statistics, I usually gloss over the details of how inference is actually performed, treating it as a black box essentially. The beauty of probabilistic programming is that you actually don't *have* to understand how the inference works in order to build models, but it certainly helps.

When I presented a new Bayesian model to Quantopian's CEO, Fawce, who wasn't trained in Bayesian stats but is eager to understand it, he started to ask about the part I usually gloss over: "Thomas, how does the inference actually work? How do we get these magical samples from the posterior?".

Now I could have said: "Well that's easy, MCMC generates samples from the posterior distribution by constructing a reversible Markov-chain that has as its equilibrium distribution the target posterior distribution. Questions?".

That statement is correct, but is it useful? My pet peeve with how math and stats are taught is that no one ever tells you about the intuition behind the concepts (which is usually quite simple) but only hands you some scary math. This is certainly the way I was taught and I had to spend countless hours banging my head against the wall until that euraka moment came about. Usually things weren't as scary or seemingly complex once I deciphered what it meant.

This blog post is an attempt at trying to explain the *intuition* behind MCMC sampling (specifically, the random-walk Metropolis algorithm). Critically, we'll be using code examples rather than formulas or math-speak. Eventually you'll need that but I personally think it's better to start with the an example and build the intuition before you move on to the math.

The problem and its unintuitive solution

Lets take a look at Bayes formula:

$$P(\theta|x) = \frac{P(x|\theta)P(\theta)}{P(x)}$$

We have $P(\theta|x)$, the probability of our model parameters θ given the data x and thus our quantity of interest. To compute this we multiply the prior $P(\theta)$ (what we think about θ before we have seen any data) and the likelihood $P(x|\theta)$, i.e. how we think our data is distributed. This nominator is pretty easy to solve for.

However, lets take a closer look at the denominator. $P(x)$ which is also called the evidence (i.e. the evidence that the data x was generated by this model). We can compute this quantity by integrating over all possible parameter values:

$$P(x) = \int_\Theta P(x, \theta) \, \mathrm{d}\theta$$

This is the key difficulty with Bayes formula – while the formula looks innocent enough, for even slightly non-trivial models you just can't compute the posterior in a closed-form way.

Now we might say "OK, if we can't solve something, could we try to approximate it? For example, if we could somehow draw samples from that posterior we can Monte Carlo approximate it." Unfortunately, to directly sample from that distribution you not only have to solve Bayes formula, but also invert it, so that's even harder.

Then we might say "Well, instead let's construct an ergodic, reversible Markov chain that has as an equilibrium distribution which matches our posterior distribution". I'm just kidding, most people wouldn't say that as it sounds bat-shit crazy. If you can't compute it, can't sample from it, then constructing that Markov chain with all these properties must be even harder.

The surprising insight though is that this is actually very easy and there exist a general class of algorithms that do this called Markov chain Monte Carlo (constructing a Markov chain to do Monte Carlo approximation).

Setting up the problem

First, lets import our modules.

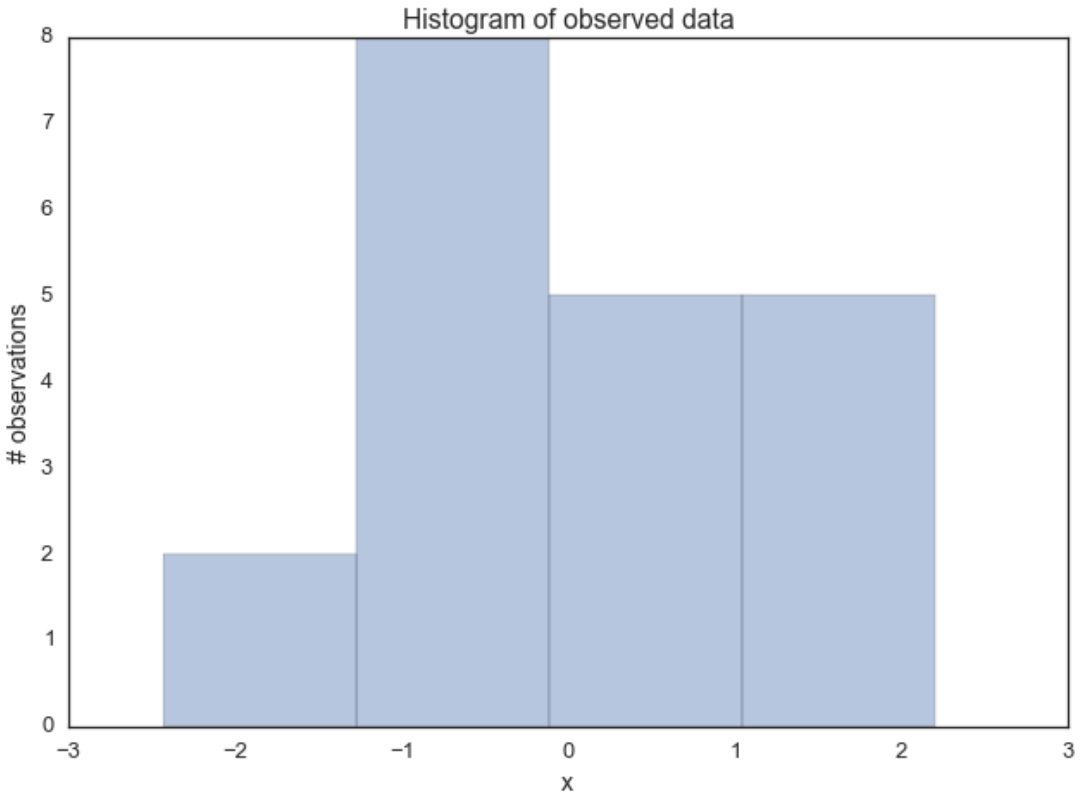
```
%matplotlib inline

import numpy as np
import scipy as sp
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from scipy.stats import norm

sns.set_style('white')
```

```
sns.set_context('talk')

np.random.seed(123)
```

Lets generate some data: 20 points from a normal centered around zero. Our goal will be to estimate the posterior of the mean `mu` (we'll assume that we know the standard deviation to be 1).

```
data = np.random.randn(20)
```

```
ax = plt.subplot()
sns.distplot(data, kde=False, ax=ax)
_ = ax.set(title='Histogram of observed data', xlabel='x', ylabel='#
```

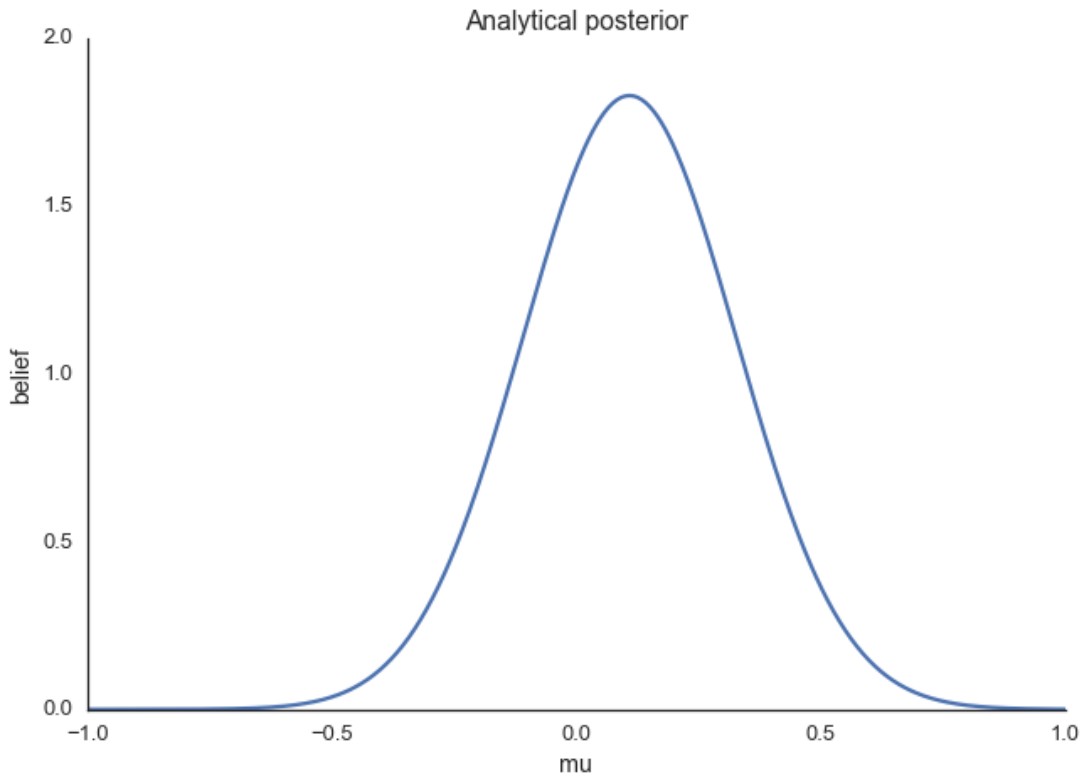


Next, we have to define our model. In this simple case, we will assume that this data is normal distributed, i.e. the likelihood of the model is normal. As you know, a normal distribution has two parameters – mean μ and standard deviation σ. For simplicity, we'll assume we know that $\sigma = 1$ and we'll want to infer the posterior for μ. For each parameter we want to infer, we have to chose a prior. For simplicity, lets also assume a Normal distribution as a prior for μ. Thus, in stats speak our model is:

$$\mu \sim \text{Normal}(0, 1) x | \mu \sim \text{Normal}(x; \mu, 1)$$

What is convenient, is that for this model, we actually can compute the posterior analytically. That's because for a normal likelihood with known standard deviation, the normal prior for `mu` is conjugate (conjugate here means that our posterior will follow the same distribution as the prior), so we know that our posterior for μ is also normal. We can easily look up on wikipedia how we can compute the parameters of the posterior. For a mathemtical derivation of this, see here.

```
def calc_posterior_analytical(data, x, mu_0, sigma_0):
    sigma = 1.
    n = len(data)
    mu_post = (mu_0 / sigma_0**2 + data.sum() / sigma**2) / (1. / sig
    sigma_post = (1. / sigma_0**2 + n / sigma**2)**-1
    return norm(mu_post, np.sqrt(sigma_post)).pdf(x)

ax = plt.subplot()
x = np.linspace(-1, 1, 500)
posterior_analytical = calc_posterior_analytical(data, x, 0., 1.)
ax.plot(x, posterior_analytical)
ax.set(xlabel='mu', ylabel='belief', title='Analytical posterior');
sns.despine()
```



This shows our quantity of interest, the probability of μ's values after having seen the data, taking our prior information into account. Lets assume, however, that our prior wasn't conjugate and we couldn't solve this by hand which is usually the case.

Explaining MCMC sampling with code

Now on to the sampling logic. At first, you find starting parameter position (can be randomly chosen), lets fix it arbitrarily to:

```
mu_current = 1.
```

Then, you propose to move (jump) from that position somewhere else (that's the Markov part). You can be very dumb or very sophisticated about how you come up with that proposal. The Metropolis sampler is very dumb and just takes a sample from a normal distribution (no relationship to the normal we assume for the model) centered around your current `mu` value (i.e. `mu_current`) with a certain standard deviation

(`proposal_width`) that will determine how far you propose jumps (here we're use `scipy.stats.norm`):

```
proposal = norm(mu_current, proposal_width).rvs()
```

Next, you evaluate whether that's a good place to jump to or not. If the resulting normal distribution with that proposed `mu` explaines the data better than your old `mu` , you'll definitely want to go there. What does "explains the data better" mean? We quantify fit by computing the probability of the data, given the likelihood (normal) with the proposed parameter values (proposed `mu` and a fixed `sigma = 1`). This can easily be computed by calculating the probability for each data point using `scipy.stats.normal(mu, sigma).pdf(data)` and then multiplying the individual probabilities, i.e. compute the likelihood (usually you would use log probabilities but we omit this here):

```
likelihood_current = norm(mu_current, 1).pdf(data).prod()
likelihood_proposal = norm(mu_proposal, 1).pdf(data).prod()

# Compute prior probability of current and proposed mu
prior_current = norm(mu_prior_mu, mu_prior_sd).pdf(mu_current)
prior_proposal = norm(mu_prior_mu, mu_prior_sd).pdf(mu_proposal)

# Nominator of Bayes formula
p_current = likelihood_current * prior_current
p_proposal = likelihood_proposal * prior_proposal
```

Up until now, we essentially have a hill-climbing algorithm that would just propose movements into random directions and only accept a jump if the `mu_proposal` has higher likelihood than `mu_current` . Eventually we'll get to `mu = 0` (or close to it) from where no more moves will be possible. However, we want to get a posterior so we'll also have to sometimes accept moves into the other direction. The key trick is by dividing the two probabilities,

```
p_accept = p_proposal / p_current
```

we get an acceptance probability. You can already see that if `p_proposal` is larger, that probability will be `> 1` and we'll definitely accept. However, if `p_current` is

larger, say twice as large, there'll be a 50% chance of moving there:

```
accept = np.random.rand() < p_accept

if accept:
    # Update position
    cur_pos = proposal
```

This simple procedure gives us samples from the posterior.

Why does this make sense?

Taking a step back, note that the above acceptance ratio is the reason this whole thing

works out and we get around the integration. We can show this by computing the acceptance ratio over the normalized posterior and seeing how it's equivalent to the acceptance ratio of the unnormalized posterior (lets say μ_0 is our current position, and μ is our proposal):

$$\frac{\frac{P(x|\mu)P(\mu)}{P(x)}}{\frac{P(x|\mu_0)P(\mu_0)}{P(x)}} = \frac{P(x|\mu)P(\mu)}{P(x|\mu_0)P(\mu_0)}$$

In words, dividing the posterior of proposed parameter setting by the posterior of the current parameter setting, $P(x)$ – that nasty quantity we can't compute – gets canceled out. So you can intuit that we're actually dividing the full posterior at one position by the full posterior at another position (no magic here). That way, we are visiting regions of high posterior probability *relatively* more often than those of low posterior probability.

Putting it all together

```
def sampler(data, samples=4, mu_init=.5, proposal_width=.5, plot=Fals
    mu_current = mu_init
    posterior = [mu_current]
    for i in range(samples):
        # suggest new position
        mu_proposal = norm(mu_current, proposal_width).rvs()

        # Compute likelihood by multiplying probabilities of each dat
        likelihood_current = norm(mu_current, 1).pdf(data).prod()
        likelihood_proposal = norm(mu_proposal, 1).pdf(data).prod()

        # Compute prior probability of current and proposed mu
        prior_current = norm(mu_prior_mu, mu_prior_sd).pdf(mu_current
        prior_proposal = norm(mu_prior_mu, mu_prior_sd).pdf(mu_propos

        p_current = likelihood_current * prior_current
        p_proposal = likelihood_proposal * prior_proposal


        # Accept proposal?
        p_accept = p_proposal / p_current

        # Usually would include prior probability, which we neglect h
        accept = np.random.rand() < p_accept

        if plot:
            plot_proposal(mu_current, mu_proposal, mu_prior_mu, mu_pr

        if accept:
            # Update position
            mu_current = mu_proposal

        posterior.append(mu_current)

    return np.array(posterior)
```

```
# Function to display
def plot_proposal(mu_current, mu_proposal, mu_prior_mu, mu_prior_sd,
    from copy import copy
    trace = copy(trace)
    fig, (ax1, ax2, ax3, ax4) = plt.subplots(ncols=4, figsize=(16, 4)
    fig.suptitle('Iteration %i' % (i + 1))
    x = np.linspace(-3, 3, 5000)
    color = 'g' if accepted else 'r'

    # Plot prior
    prior_current = norm(mu_prior_mu, mu_prior_sd).pdf(mu_current)
    prior_proposal = norm(mu_prior_mu, mu_prior_sd).pdf(mu_proposal)
    prior = norm(mu_prior_mu, mu_prior_sd).pdf(x)
    ax1.plot(x, prior)
    ax1.plot([mu_current] * 2, [0, prior_current], marker='o', color=
    ax1.plot([mu_proposal] * 2, [0, prior_proposal], marker='o', colo
    ax1.annotate("", xy=(mu_proposal, 0.2), xytext=(mu_current, 0.2),
                 arrowprops=dict(arrowstyle="->", lw=2.))
    ax1.set(ylabel='Probability Density', title='current: prior(mu=%

    # Likelihood
    likelihood_current = norm(mu_current, 1).pdf(data).prod()
    likelihood_proposal = norm(mu_proposal, 1).pdf(data).prod()
    y = norm(loc=mu_proposal, scale=1).pdf(x)
    sns.distplot(data, kde=False, norm_hist=True, ax=ax2)
    ax2.plot(x, y, color=color)
    ax2.axvline(mu_current, color='b', linestyle='--', label='mu_curr
    ax2.axvline(mu_proposal, color=color, linestyle='--', label='mu_p
    #ax2.title('Proposal {}'.format('accepted' if accepted else 'reje
    ax2.annotate("", xy=(mu_proposal, 0.2), xytext=(mu_current, 0.2),
                 arrowprops=dict(arrowstyle="->", lw=2.))
    ax2.set(title='likelihood(mu=%.2f) = %.2f\nlikelihood(mu=%.2f) =

    # Posterior
    posterior_analytical = calc_posterior_analytical(data, x, mu_prio
    ax3.plot(x, posterior_analytical)
    posterior_current = calc_posterior_analytical(data, mu_current, m
    posterior_proposal = calc_posterior_analytical(data, mu_proposal,
    ax3.plot([mu_current] * 2, [0, posterior_current], marker='o', co
    ax3.plot([mu_proposal] * 2, [0, posterior_proposal], marker='o',
    ax3.annotate("", xy=(mu_proposal, 0.2), xytext=(mu_current, 0.2),
                 arrowprops=dict(arrowstyle="->", lw=2.))
    #x3.set(title=r'prior x likelihood $\propto$ posterior')
    ax3.set(title='posterior(mu=%.2f) = %.5f\nposterior(mu=%.2f) = %

    if accepted:
        trace.append(mu_proposal)
    else:
        trace.append(mu_current)
    ax4.plot(trace)
    ax4.set(xlabel='iteration', ylabel='mu', title='trace')
    plt.tight_layout()
    #plt.legend()
```

Visualizing MCMC

To visualize the sampling, we'll create plots for some quantities that are computed. Each row below is a single iteration through our Metropolis sampler.

The first columns is our prior distribution – what our belief about μ is before seeing the data. You can see how the distribution is static and we only plug in our μ proposals. The vertical lines represent our current μ in blue and our proposed μ in either red or green (rejected or accepted, respectively).

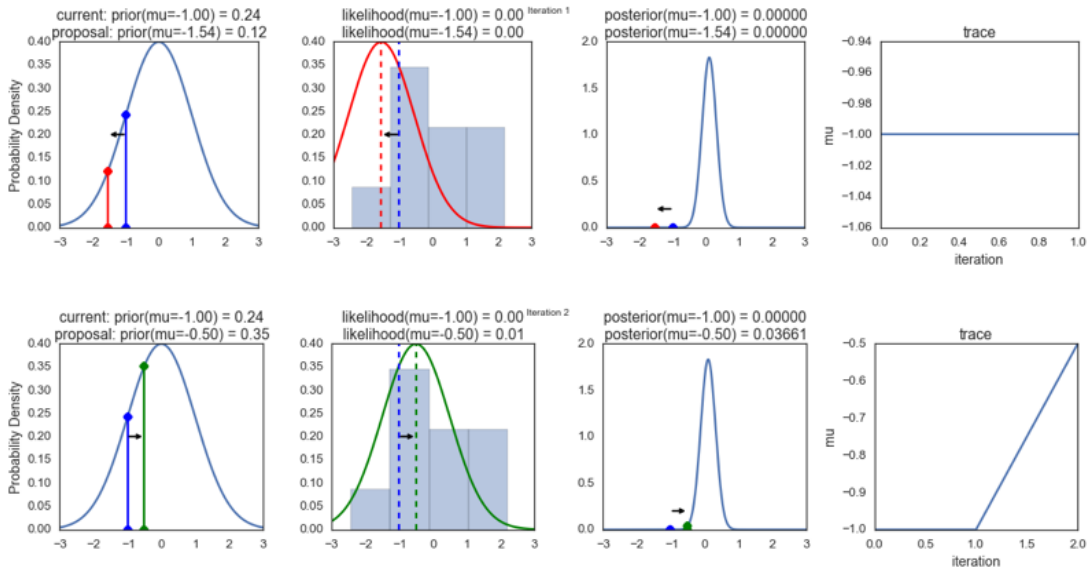
The 2nd column is our likelihood and what we are using to evaluate how good our model explains the data. You can see that the likelihood function changes in response to the proposed μ. The blue histogram which is our data. The solid line in green or red is the likelihood with the currently proposed `mu`. Intuitively, the more overlap there is between likelihood and data, the better the model explains the data and the higher the resulting probability will be. The dotted line of the same color is the proposed `mu` and the dotted blue line is the current `mu`.

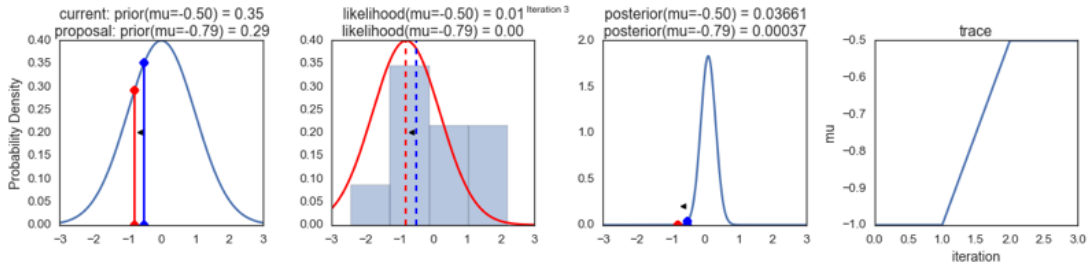
The 3rd column is our posterior distribution. Here I am displaying the normalized posterior but as we found out above, we can just multiply the prior value for the current and proposed μ's by the likelihood value for the two μ's to get the unnormalized posterior values (which we use for the actual computation), and divide one by the other to get our acceptance probability.

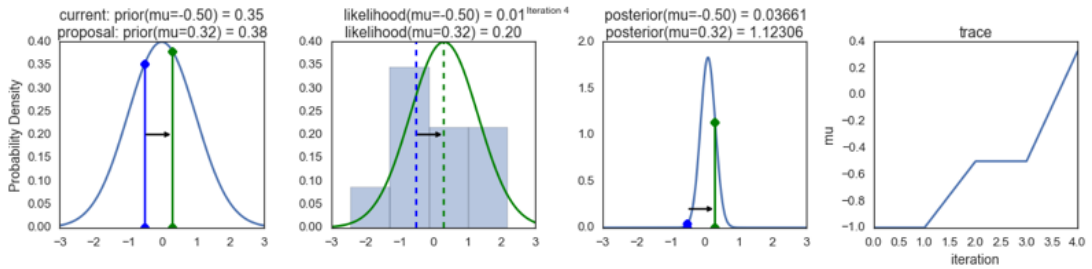
The 4th column is our trace (i.e. the posterior samples of μ we're generating) where we store each sample irrespective of whether it was accepted or rejected (in which case the line just stays constant).

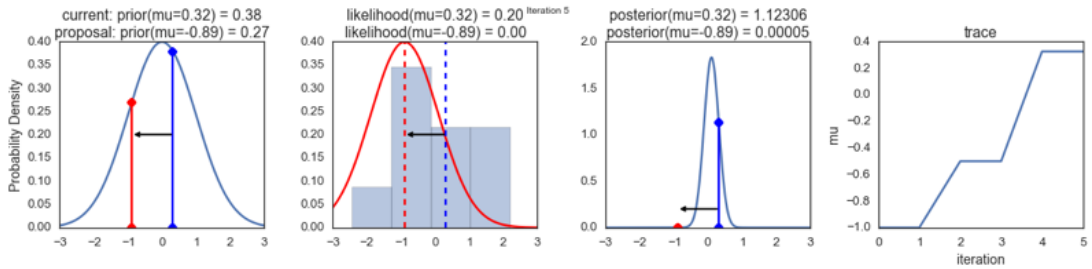
Note that we always move to relatively more likely μ values (in terms of their posterior density), but only sometimes to relatively less likely μ values, as can be seen in iteration 14 (the iteration number can be found at the top center of each row).

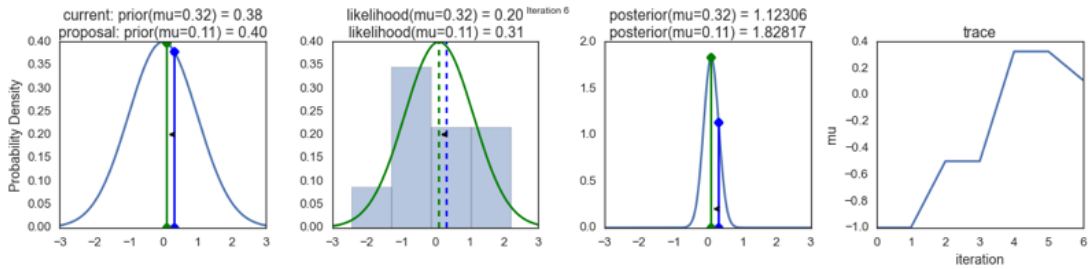
```
np.random.seed(123)
sampler(data, samples=8, mu_init=-1., plot=True);
```

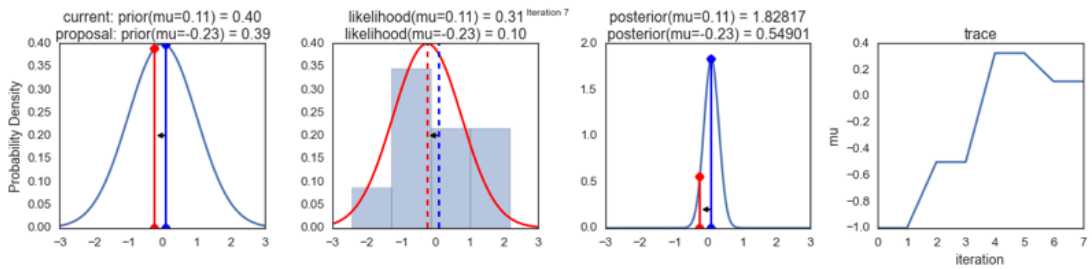


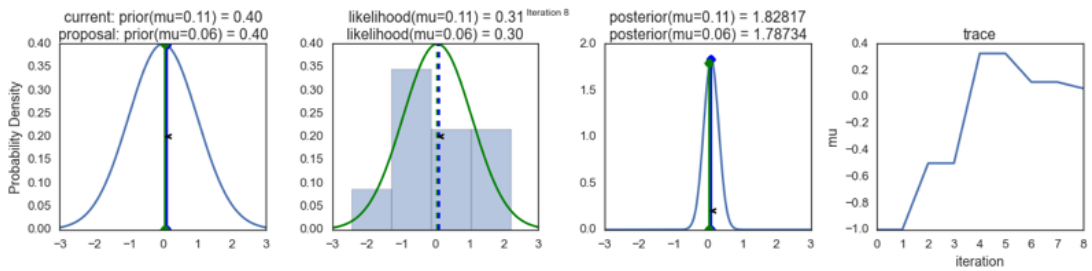










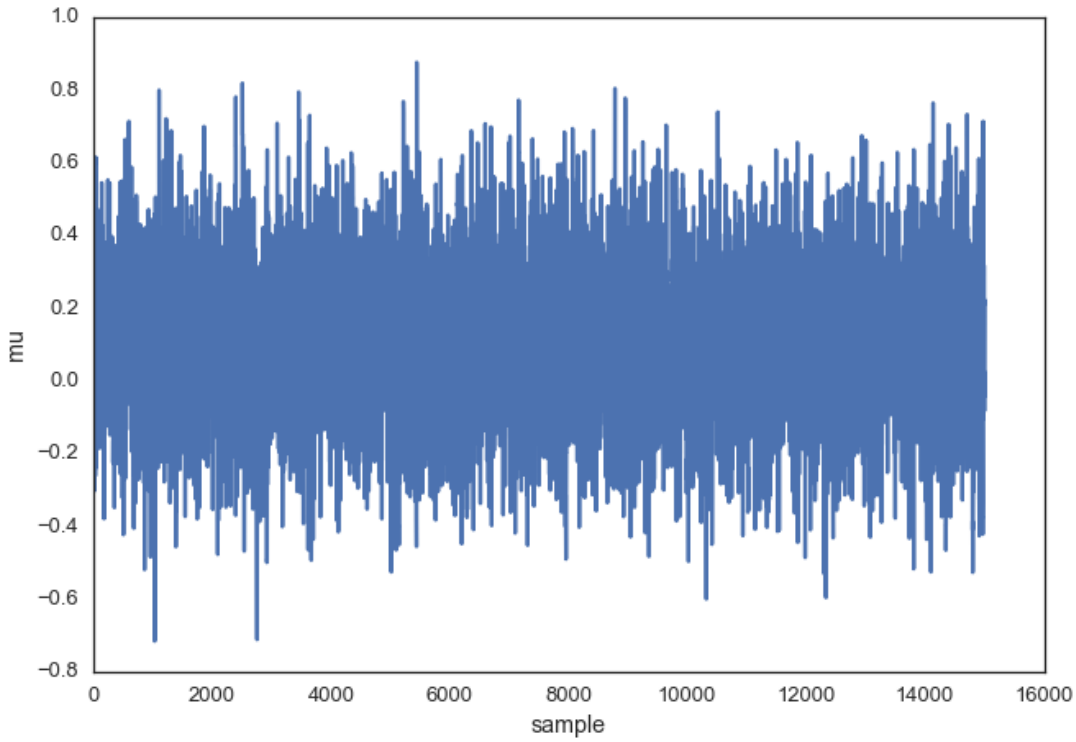


Now the magic of MCMC is that you just have to do that for a long time, and the

samples that are generated in this way come from the posterior distribution of your model. There is a rigorous mathematical proof that guarantees this which I won't go into detail here.

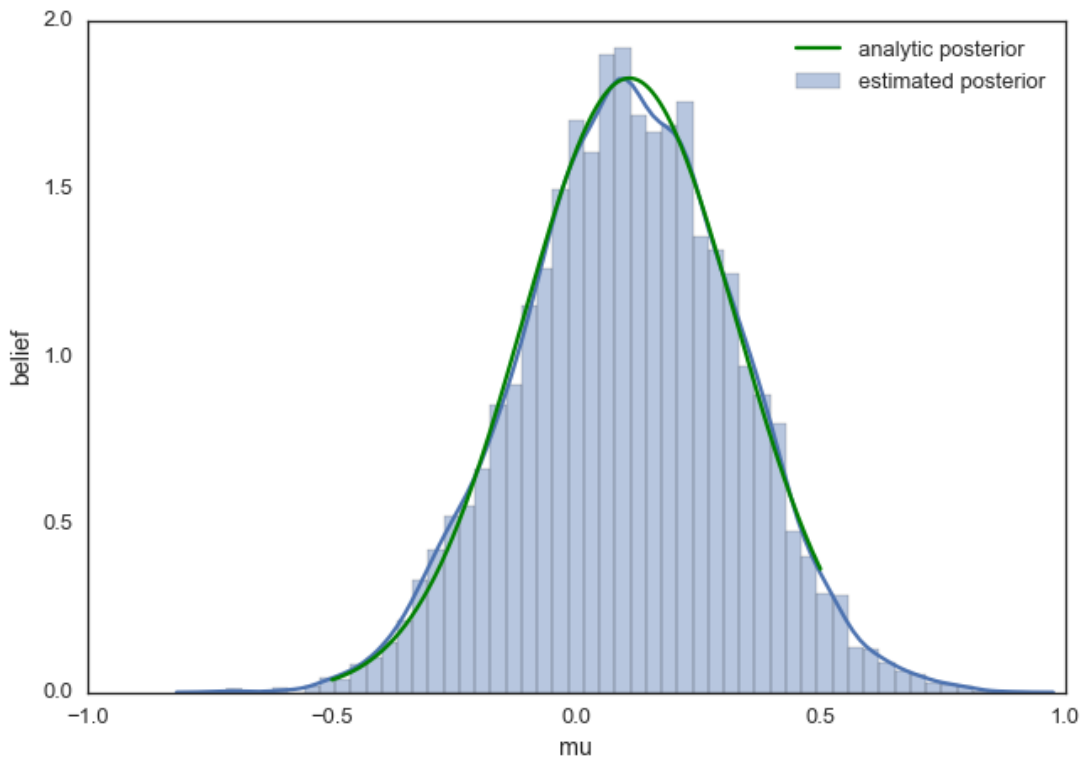
To get a sense of what this produces, lets draw a lot of samples and plot them.

```
posterior = sampler(data, samples=15000, mu_init=1.)
fig, ax = plt.subplots()
ax.plot(posterior)
_ = ax.set(xlabel='sample', ylabel='mu');
```



This is usually called the trace. To now get an approximation of the posterior (the reason why we're doing all this), we simply take the histogram of this trace. It's important to keep in mind that although this looks similar to the data we sampled above to fit the model, the two are completely separate. The below plot represents our **belief** in `mu`. In this case it just happens to also be normal but for a different model, it could have a completely different shape than the likelihood or prior.

```
ax = plt.subplot()

sns.distplot(posterior[500:], ax=ax, label='estimated posterior')
x = np.linspace(-.5, .5, 500)
post = calc_posterior_analytical(data, x, 0, 1)
ax.plot(x, post, 'g', label='analytic posterior')
_ = ax.set(xlabel='mu', ylabel='belief');
ax.legend();
```



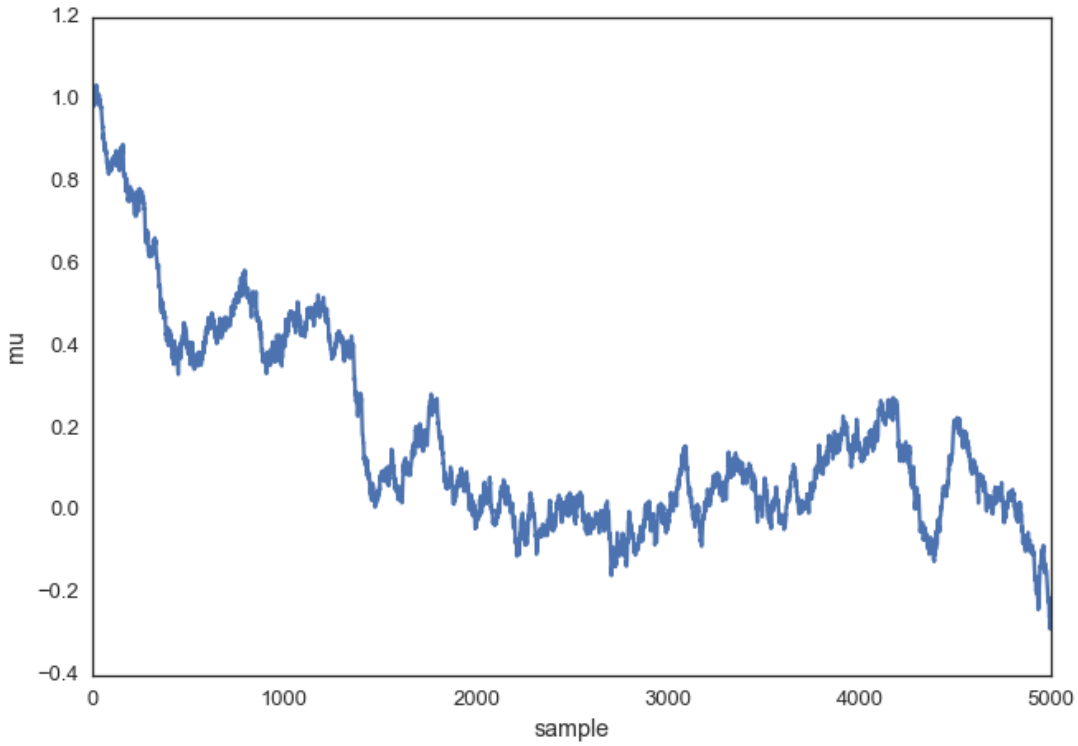
As you can see, by following the above procedure, we get samples from the same distribution as what we derived analytically.

Proposal width

Above we set the proposal width to `0.5` . That turned out to be a pretty good value. In general you don't want the width to be too narrow because your sampling will be

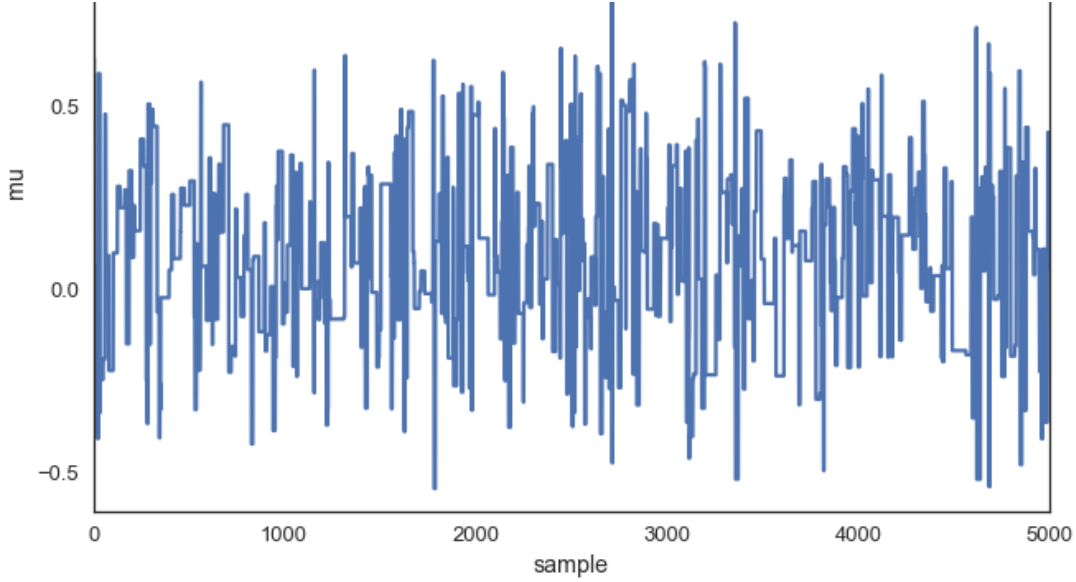
inefficient as it takes a long time to explore the whole parameter space and shows the typical random-walk behavior:

```
posterior_small = sampler(data, samples=5000, mu_init=1., proposal_w
fig, ax = plt.subplots()
ax.plot(posterior_small);
_ = ax.set(xlabel='sample', ylabel='mu');
```



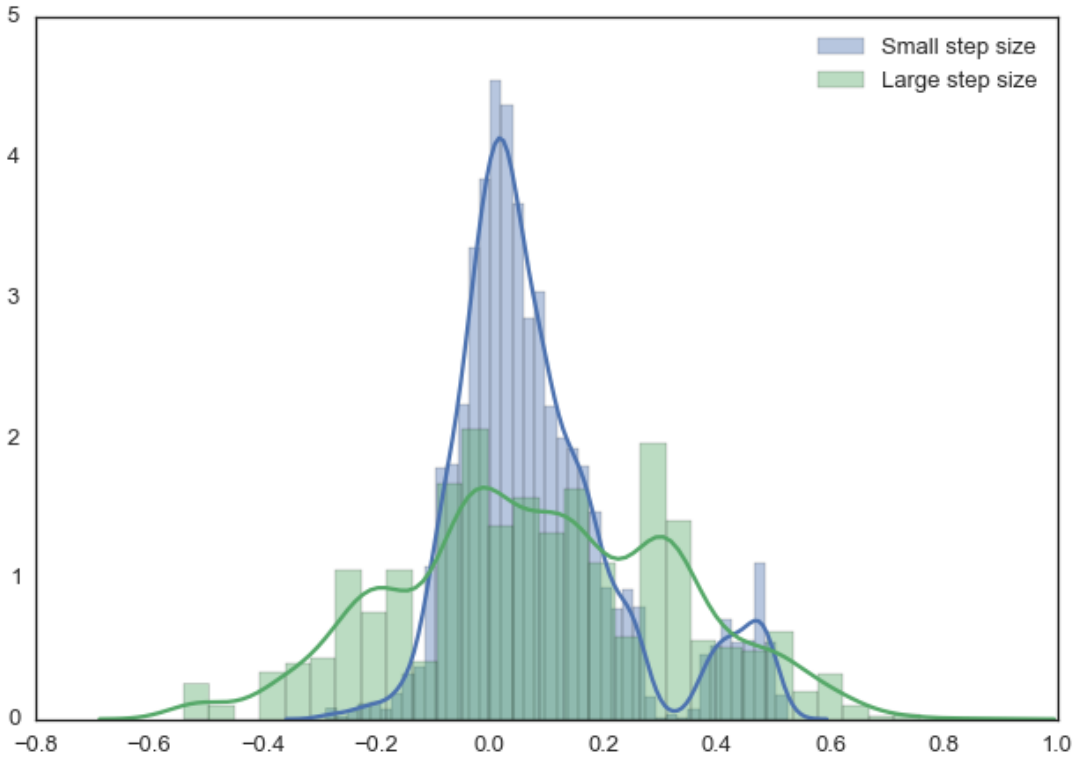
But you also don't want it to be so large that you never accept a jump:

```
posterior_large = sampler(data, samples=5000, mu_init=1., proposal_w
fig, ax = plt.subplots()
ax.plot(posterior_large); plt.xlabel('sample'); plt.ylabel('mu');
_ = ax.set(xlabel='sample', ylabel='mu');
```



Note, however, that we are still sampling from our target posterior distribution here as guaranteed by the mathematical proof, just less efficiently:

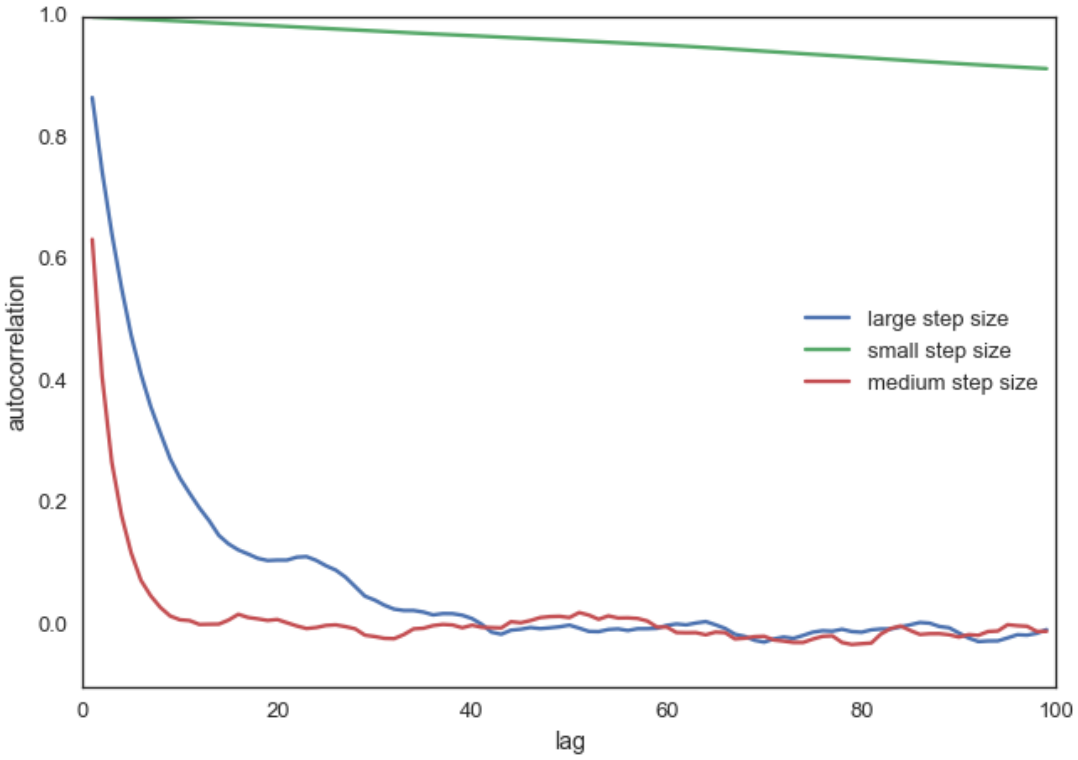
```
sns.distplot(posterior_small[1000:], label='Small step size')
sns.distplot(posterior_large[1000:], label='Large step size');
_ = plt.legend();
```



With more samples this will eventually look like the true posterior. The key is that we want our samples to be independent of each other which cleary isn't the case here. Thus, one common metric to evaluate the efficiency of our sampler is the autocorrelation – i.e. how correlated a sample `i` is to sample `i-1`, `i-2`, etc:

```
from pymc3.stats import autocorr
lags = np.arange(1, 100)
fig, ax = plt.subplots()
ax.plot(lags, [autocorr(posterior_large, l) for l in lags], label='la
ax.plot(lags, [autocorr(posterior_small, l) for l in lags], label='sr
```

```
ax.plot(lags, [autocorr(posterior, l) for l in lags], label='medium s
ax.legend(loc=0)
_ = ax.set(xlabel='lag', ylabel='autocorrelation', ylim=(-.1, 1))
```



Obviously we want to have a smart way of figuring out the right step width automatically. One common method is to keep adjusting the proposal width so that roughly 50% proposals are rejected.

Extending to more complex models

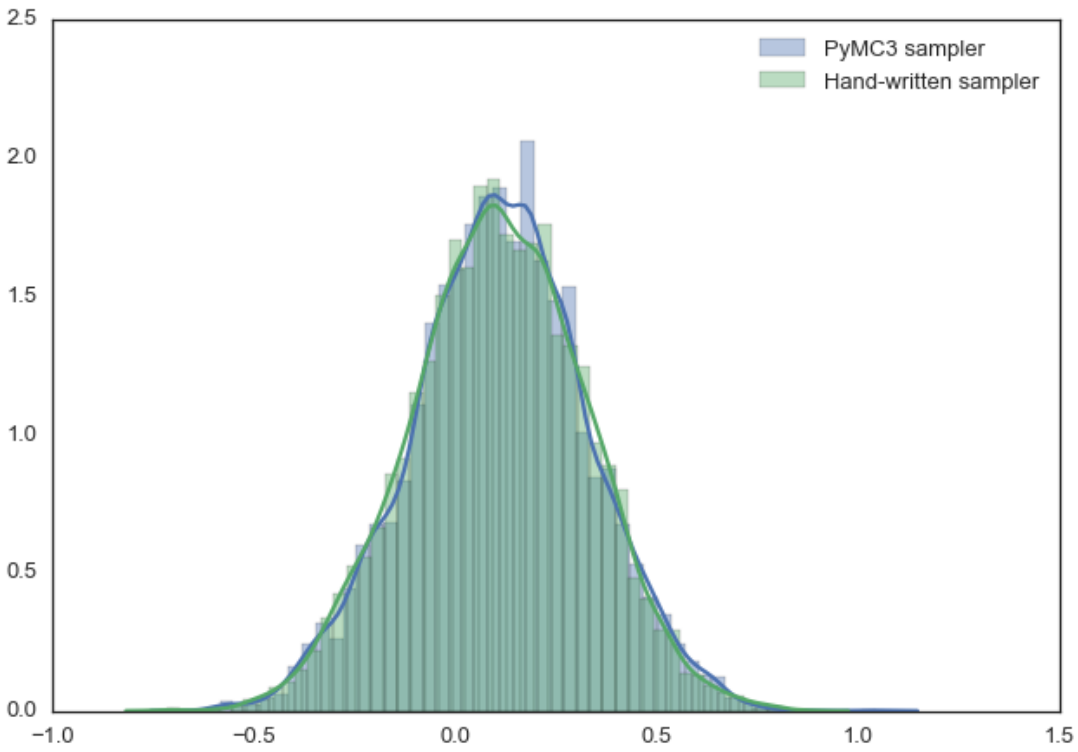
Now you can easily imagine that we could also add a `sigma` parameter for the standard-deviation and follow the same procedure for this second parameter. In that case, we would be generating proposals for `mu` *and* `sigma` but the algorithm logic would be nearly identical. Or, we could have data from a very different distribution like a Binomial and still use the same algorithm and get the correct posterior. That's pretty cool and a huge benefit of probabilistic programming: Just define the model you want and let MCMC take care of the inference.

For example, the below model can be written in `PyMC3` quite easily. Below we also use

the Metropolis sampler (which automatically tunes the proposal width) and see that we get identical results. Feel free to play around with this and change the distributions. For more information, as well as more complex examples, see the PyMC3 documentation.

```
import pymc3 as pm

with pm.Model():
    mu = pm.Normal('mu', 0, 1)
    sigma = 1.
    returns = pm.Normal('returns', mu=mu, sd=sigma, observed=data)

    step = pm.Metropolis()

    trace = pm.sample(15000, step)

sns.distplot(trace[2000:]['mu'], label='PyMC3 sampler');
sns.distplot(posterior[500:], label='Hand-written sampler');
plt.legend();
```

```
 [-----------------100%-----------------] 15000 of 15000 complete in
1.7 sec
```



Conclusions

We glossed over a lot of detail which is certainly important but there are many other posts that deal with that. Here, we really wanted to communicate the idea of MCMC and the Metropolis sampler. Hopefully you will have gathered some intuition which will equip you to read one of the more technical introductions to this topic.

Other, more fancy, MCMC algorithms like Hamiltonian Monte Carlo actually work very similar to this, they are just much more clever in proposing where to jump next.

This blog post was written in a Jupyter Notebook, you can find the underlying NB with all its code here.

Support me on Patreon

Finally, if you enjoyed this blog post, consider supporting me on Patreon which allows me to devote more time to writing new blog posts.

7 0 Reply

Thomas Wiecki Mod → Brandon Rohrer

10 years ago

Thanks, Brandon. Glad I'm not the only one who finds it useful.

2 0 Reply

G

gewinnste

9 years ago

THIS is for dummies? I'm gonna get a rope ...

17 3 Reply

rihardsbar → gewinnste

6 years ago

It's very intuitive, uses code and graphs, cant get simplier than that, rather than using lots of fancy maths.

2 0 Reply

H

huts

9 years ago

Best article I have read so far to understand MCMC. Thank you so much Thomas! I was wondering: could you make another article explaining where the Markov Chain is involved :p?

4 0 Reply

Lefkios C. Geladaris → huts

6 years ago edited

It's exactly this line of code here:
if accept:
# Update position
mu_current = mu_proposal # Markov Assumption -> No memory

You only keep the proposed mu, no history. So basically the Markov part is just an assumption to simplify the whole process. In more detail, you are discarding the current mu for the new one every time if you accept. So at any point in time you depend on your current mu, not in its history i.e. Markov assumption.

0 0 Reply

S

sardhendu mishra

9 years ago

Thanks Thomas. This is one of the best post I have come across. Thank you so much

2 0 Reply

Thomas Wiecki Mod → sardhendu mishra

9 years ago

Thanks!

0 0 Reply

Brijendra Pratap Singh Janwaar

8 years ago

Excellent writing. Finally I understood the MCMC. Thanks a lot.

1 0 Reply

Robert M. Johnson

8 years ago

If all algorithms were explained as clearly as this, I'd be so much further ahead in having all of these statistical tools in my mental toolbox. Thank you so much for taking the time to write this!

1 0 Reply

Thomas Wiecki Mod → Robert M. Johnson

8 years ago

Thanks for your comment!

4 0 Reply

janeXXX

4 years ago

Hi and thanks for the great introduction. I don't know if you still update the page, but I wanted to ask you smt. Could you explain what "mean of m samples" and "mean of c samples" stand for? I'm trying to find the convergence of my chain and I'm kind of lost in terminology. I can see on the plots that the different parameters are converging but I cannot figure what is the measure for the whole chain with all parameters.

0 0 Reply

R

Rishi Kambhampati

4 years ago edited

Hello, sorry if my question is naive, but what does "Our goal will be to estimate the posterior of the mean mu (we'll assume that we know the standard deviation to be 1)" mean? If we take a real world example where we fit the model with data and we have 10 features with 10 coefficients, how can we draw the analogy between the said example and the real world case. What exactly are we is our goal? to calculate the mean of the coefficient of each feature? How do you calculate likelihood for data points in real situations?Thanks in advance.

0 0 Reply

Thomas Wiecki Mod → Rishi Kambhampati

4 years ago

Hey Rishi,

You can imagine how you would fit this in a non-Bayesian way. You'd just estimate the values for the 10 coefficients, right? But there's uncertainty around those coefficients, the estimates aren't exact. But your point-estimate is not conveying that uncertainty. In Bayesian modeling we thus estimate a posterior distribution rather than a point-estimate. So you'd get 10 posteriors for 10 coefficients.

If you want to learn more about the basics, check out www.intuitivebayes.com where we go through all of this.

0 0 Reply

A

anand Hegde

4 years ago

Great article !

0 0 Reply

J

John

5 years ago

this might be obvious, but for the first bell shaped curve shown, can you explain 'belief' as the y axis?
I guess I am expecting a probability? What is the unit of belief? the mean? I cannot wrap my head around this.

0 0 Reply

Thomas Wiecki Mod → John

5 years ago

It's probability density. I often put belief as it's more intuitive for newcomers.

0 0 Reply

J

John → Thomas Wiecki

5 years ago

Thanks Thomas, I understand the basic gist of this, and is much appreciated. I do have a follow-up . Where you have four columns, for instance, the first one, you have prior=.24. that looks like a probability, but the probability of a single point is 0, is it not? so, not sure what the .24 means. Also, the proposal is based on new information, correct? Why is prior mentioned again? Is that your new (posterior) prediction about the mean? probability =0.12?

thanks again

0 0 Reply

Thomas Wiecki Mod

John

4 years ago

The probability mass is zero, but not the density (which is what I'm displaying here). At every step, we need the prior and the likelihood to compute the (unnormalized) posterior value which we then use for the decision whether to jump or not.

1 1 Reply

A

Aditya Pandey

5 years ago

If one can somehow obtain the denominator (i.e. evidence), do we not need to sample or would you say that there are other benefits as well?

0 0 Reply

S

Surya Krishnamurthy

6 years ago

Brilliant post!

0 0 Reply

David Bridgeland

6 years ago

Superb, particularly the 4x8 step-by-step illustration of Metropolis sampling.

0 0 Reply

Xiaomi Yang

6 years ago edited

Hi Thomas, thank you very much for your fantastic explaining for MCMC. May I ask which software is for the code? Sorry... I see now it's Python!

0 0 Reply

D

Dawid

6 years ago edited

Hi Thomas, thank you for a great tutorial! This is incredible help in understanding MCMC.

However, I still do not understand why do we really need to build a Markov Chain for the inference. In the beginning of your post you write

we cannot solve the Bayes formula due to the P(x) term in the denominator. However, why do we need to calculate it at all? After all that's just a normalizing constant so why can't we just solve the numerator, draw some samples from it, and then normalize it to have unit mass? I'm not sure if I can explain clearly what I mean so I just paste code how I computed the posterior in your example without simulation. What's wrong with this approach? Does it have problems with highly dimensional priors/posteriors ?

```
mu_prior_mu=0
mu_prior_sd=1.

my_posterior = list()

mu_is = np.linspace(-3, 3, 601)
for mu_i in mu_is:
prior_i = norm(mu_prior_mu, mu_prior_sd).pdf(mu_i)
likelihood_i = norm(mu_i, 1).pdf(data).prod()
my_posterior.append(likelihood_i * prior_i)

# Calculate normalizing constant:
my_posterior_sum = sum(my_posterior)/100 # divide by 100 since
every bin in the prior histogram (prior_i) has the width of 0.01

# Final posterior is:
my_posterior / my_posterior_sum
```

Attached my posterior compared to the ones from the blogpost.

0 0 Reply

Thomas Wiecki Mod → Dawid

6 years ago

Yes, you can certainly do that, you're still integrating though. It's called the grid method and it works just fine for this simple example, but now try to do that even on a 5 or 10 parameter

model and you'll run into problems very quickly -- the curse of dimensionality where the space you're integrating grows exponentially with the number of dimensions. This is the key motivation for MCMC compared to most other integration methods -- it scales better with increasing dimensionality.

0 0 Reply

D **Dawid** → Thomas Wiecki
6 years ago

Great! Thank you very much Thomas for clarifying this!

0 0 Reply

P **person**
6 years ago

this was amazing; thank you!!!!

0 0 Reply

A **Ayush Sahu**
6 years ago

Amazingly well written. I could not wrap my head around these heavy terms at all, but turns out they are quite simple and intuitive.

0 0 Reply

 **Go_FreeMarkets**
6 years ago

Humanity thanks you

0 0 Reply

C **caiusCaligula**
6 years ago

Nominator?

https://math.stackexchange....

0 0 Reply

 **Thomas Wiecki** Mod → caiusCaligula
6 years ago

Thanks :)

1 0 Reply

Z **Zhibin Li**
6 years ago

This is a great starter pack!!!

0 0 Reply

kushi

6 years ago

I loved this post and I was able to understand it well enough to code it in R. However, as an exercise I wanted to try the slightly more complicated model, where even the standard deviation is unknown. I assume a prior distribution, and start with a current guess value. I evaluate the likelihood of the data for the current and proposed values of mu and sd. Then I am able to compute the prior probabilities of current and proposed values of mu and sd, i.e. I have four set of values prior_mu_current, prior_mu_proposal, prior_sd_current, prior_sd_proposal. After this I am lost as to how to compute the p_current and p_posterior. I know this is a stupid question, but any insight would be helpful.

1) Do we just use, for example, p_current = likelihood_current x prior_mu_current x prior_sd_current. Similarly, for p_proposed?

2) If the above step is correct, then after computing p_accept, do we always accept or reject values in pairs?

0 0 Reply

M

Mayukh Mukhopadhyay

7 years ago edited

Your blog posts are gold mine of understanding intricate concepts. Thanks a gogol! Can you please
write a hands-on tutorial on Ramanujan's mock theta functions for layman like me.

0 0 Reply

G

Gledson Melotti

7 years ago

Hi, how are you? One more post from you that is incredible. Very good to accompany your explanations. I have a doubt. I tried to apply this code to my datas et and realized that the code does not update posterior. It always remains with the same initial value. However, when I use a data set created with date = np.random.randn (20) the code works.
Could you tell me what be happening so I will not update the posterior with my data set?
I thank you for your attention.

0 0 Reply

J

Jae Duk Seo

7 years ago

amazing

0 0 Reply

A **Alexander Whyte**

7 years ago

This is a really good post.

0 0 Reply

 **VTinos**

7 years ago

Hi, without reading further, I don't understand the following sentence at the beginning:

"Unfortunately, to directly sample from that distribution you not only have to solve Bayes formula, but also invert it, so that's even harder."

Could you develop?

Thanks!!

0 0 Reply

 **Thomas Wiecki** Mod → VTinos

7 years ago

You can disregard that, I probably shouldn't have mentioned that. You can sample from any distribution by inverting the CDF. Sounds simple, but if you wanted to do that on any arbitrary Bayesian posterior it would be extremely difficult. The MCMC thing sounds like it should be very difficult but is actually very simple.

1 0 Reply

 **VTinos** → Thomas Wiecki

7 years ago

Thought more about it. Digged into it. Now got it. Thanks!

0 0 Reply

G **Gabriele T.**

7 years ago

Great post and tutorial. I am trying to get my head around something: is it possible to use MCMC to determine the actual distribution from where data was extracted from? So, in connection to the example above, the data consists of N= 20 points extracted from a normal distribution, what I would like to have is data* with N-> inf, such that data and data* come from the same distribution. Is it possible to design a MCMC such that with many steps the distribution resulting from MCMC would converge to this data*? If so, could you suggest

how or if you know methods suited for that, if MCMC is not the right tool? Thanks!

0 0 Reply

Thomas Wiecki Mod → Gabriele T.

7 years ago

So rather than defining a likelihood, you want to infer the likelihood, non-parameterically from data? You will certainly have to make *some* assumptions about your data. What you could do is run a Dirichlet Process Gaussian Mixture model which would be similar to a KDE: https://docs.pymc.io/notebo...

This is not a very common thing to do as you usually have some knowledge about the distribution of your data, but without knowing more about your problem I can't comment further.

0 0 Reply

G

Gabriele T. → Thomas Wiecki

7 years ago edited

Hi, Thanks a lot for the insight. Although I have information about the data, it's quite complex. However, as a first attempt I will try to see what happens using a gaussian distribution and estimate an error from this. I will certainly look more into dirichlet process gaussian mixtures.

Sorry to bother, I have another basic but related question. If I want to add a sigma to the mcmc example above, the most important modifications to the code above should be that now one has to also include also a draw for the proposed sigma, and then the likelihood should contain both proposed sigma and proposed mu, does this make sense? So for the proposed sigma

sigma_proposal = invgamma(alpha,beta).rvs())

And then the likelihood for the proposed sigma and mu becomes

likelihood_proposal = norm(mu_proposal,


```
while my_mcmc:
  gently(samples)
```